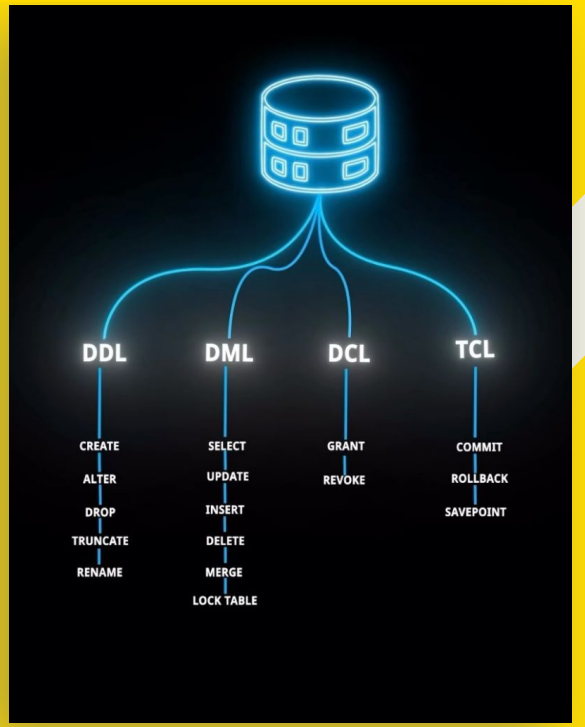
Midterm Review Workshop



Contents

01 PYTHON DATA STRUCTURE — Slide 03

02 ERD, CARDINALITY, NORMALIZATION — Slide 05

03 CREATE AND INSERT (DDL/DML) — Slide 06

04 SELECT WITH FILTER (DML) — Slide 15

05 JOIN (DML) — Slide 16

06 SUBQUERY-JOIN-UPDATE — Slide 20

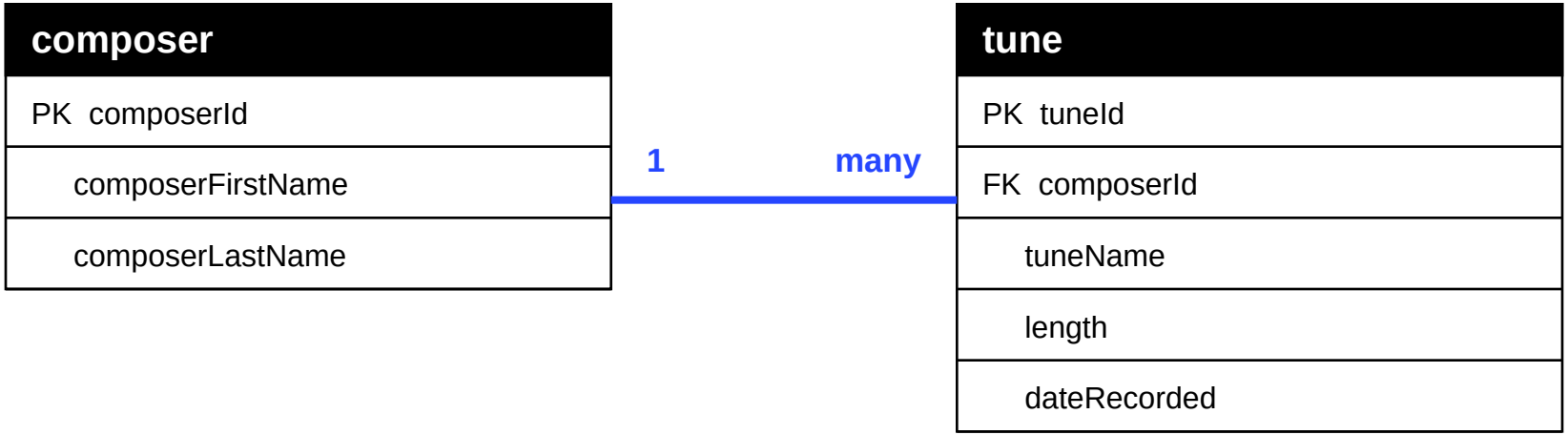
The same music data in Python

```
music = [
    {'tuneId': 1, 'tuneName': 'Moonlight Sonata',
     'length': 15, 'dateRecorded': '1801-01-01',
     'composerId': 1, 'composerFirstName': 'Ludwig',
     'composerLastName': 'Beethoven'},
    {'tuneId': 2, 'tuneName': 'Clair de Lune',
     'length': 5, 'dateRecorded': '1905-01-01',
     'composerId': 2, 'composerFirstName': 'Claude',
     'composerLastName': 'Debussy'},
    {'tuneId': 3, 'tuneName': 'Fur Elise',
     'length': 3, 'dateRecorded': '1810-01-01',
     'composerId': 1, 'composerFirstName': 'Ludwig',
     'composerLastName': 'Beethoven'}
]
```

Reading the Python data structure

1. What is the outer data structure?
2. What does one dictionary represent?
3. Which items are keys? Which items are values?
4. Which values identify a tune and a composer?
5. What composer data repeats across records?
6. What problem could that repetition cause during an update?

From repeated records to related tables



Discuss before writing SQL

1. What is the cardinality between composer and tune?
2. What normal form are these tables in? Explain your answer.
3. Which key connects the two tables?

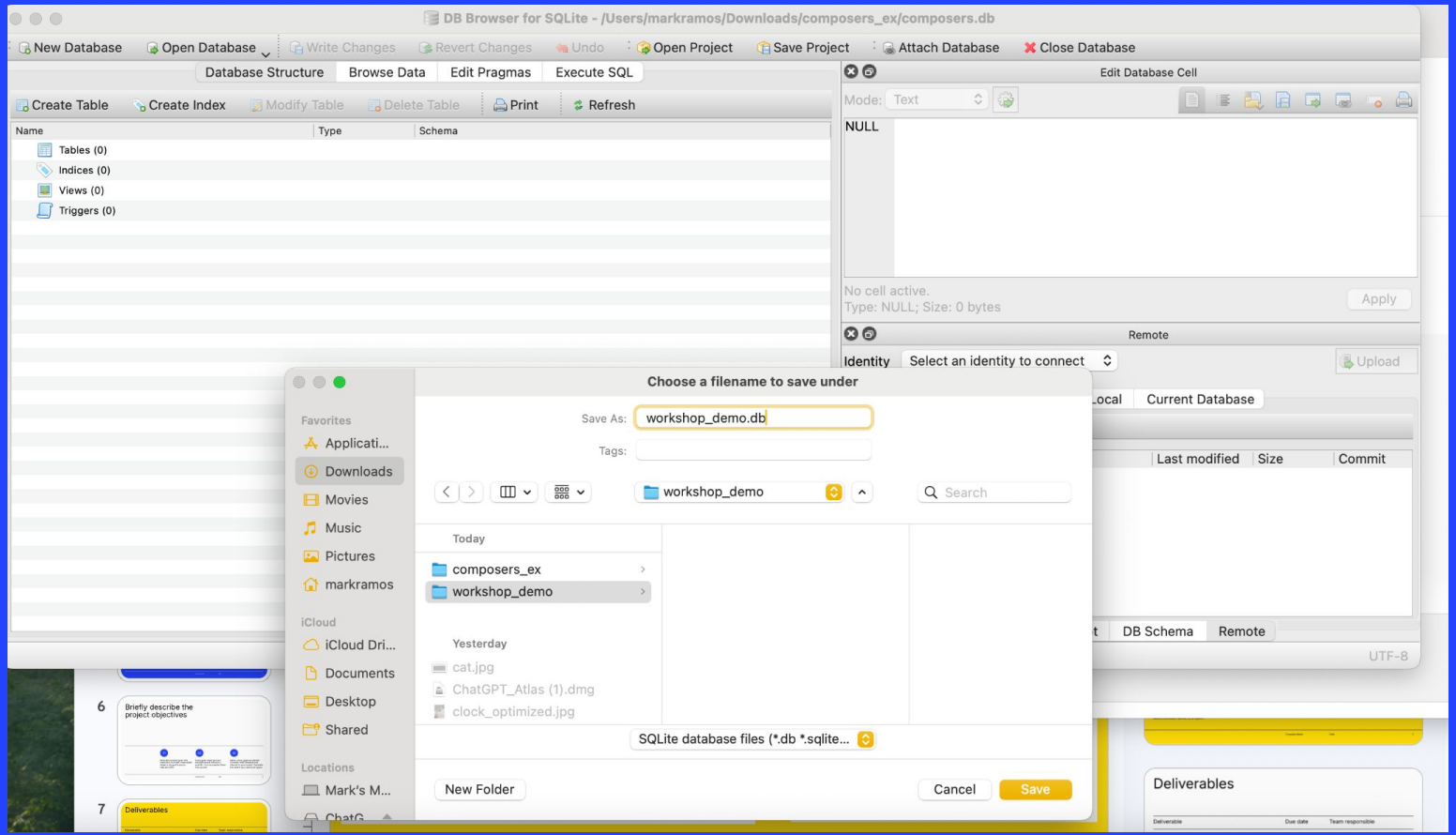
Two DDL methods (your choice):

GENERAL SQLITE CREATE TABLE SYNTAX:

```
>>sqlite3 nameofdb.db
>> PRAGMA foreign_keys = ON;
>> CREATE TABLE table1 (
  table1Id INTEGER PRIMARY KEY AUTOINCREMENT,
  table1Text TEXT NOT NULL,
  table1Value INTEGER NOT NULL CHECK (table1Value > 1),
  table1ForeignKey INTEGER,
 FOREIGN KEY (table1ForeignKey) REFERENCES table2(table2PrimaryKey)
);
```

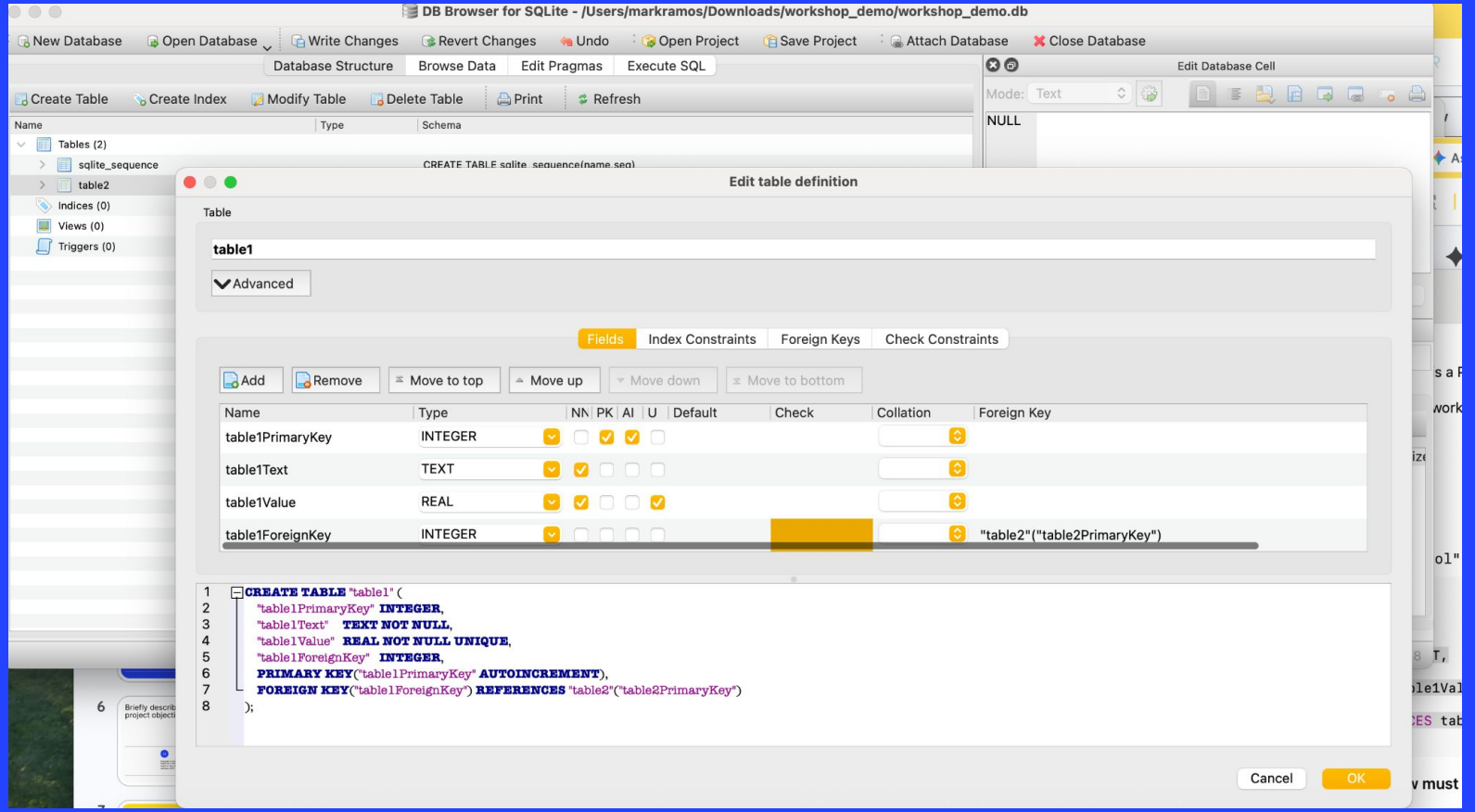
DBSqlite

New Database



DBSqlite:

Create Table



Create a database with the following two tables and populate it with data:

Create a music database: 01

Create a **tune** table containing:

- tuneId (PK)
- composerId (FK)
- tuneName
- length (# of mins)
- dateRecorded

```
sqlite> INSERT INTO tableName
(column1, column2, column3)
   ...> VALUES (value1, value2, value3);
```

Create a **composer** table containing (could be a musician w/ first name ,last name):
composerId (PK)
composerFirstName
composerLastName

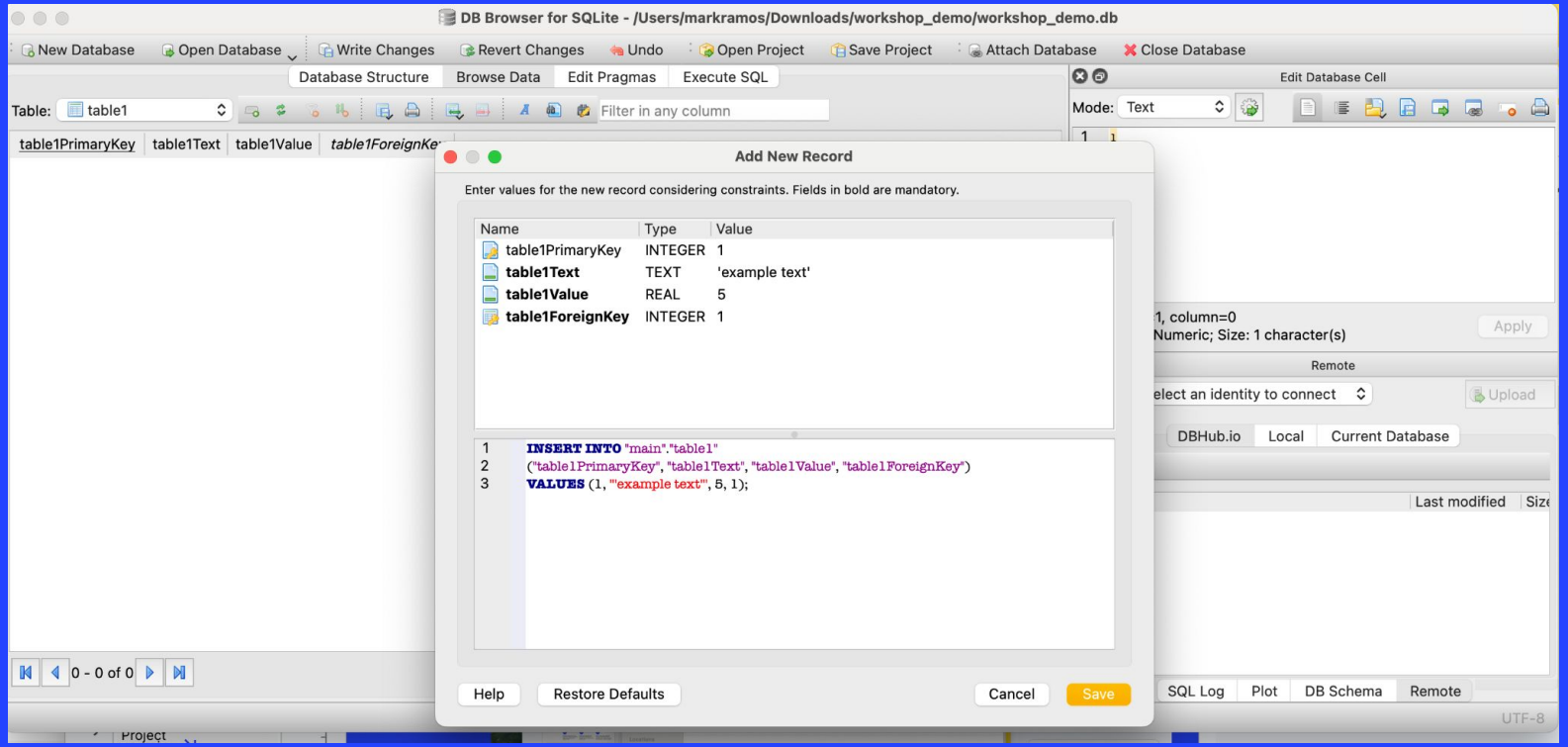
Populate it with data: 02

- Insert **5 rows** into the composer table
  You can choose the names
- Insert **5 rows** into the tune table
  You can choose the songs

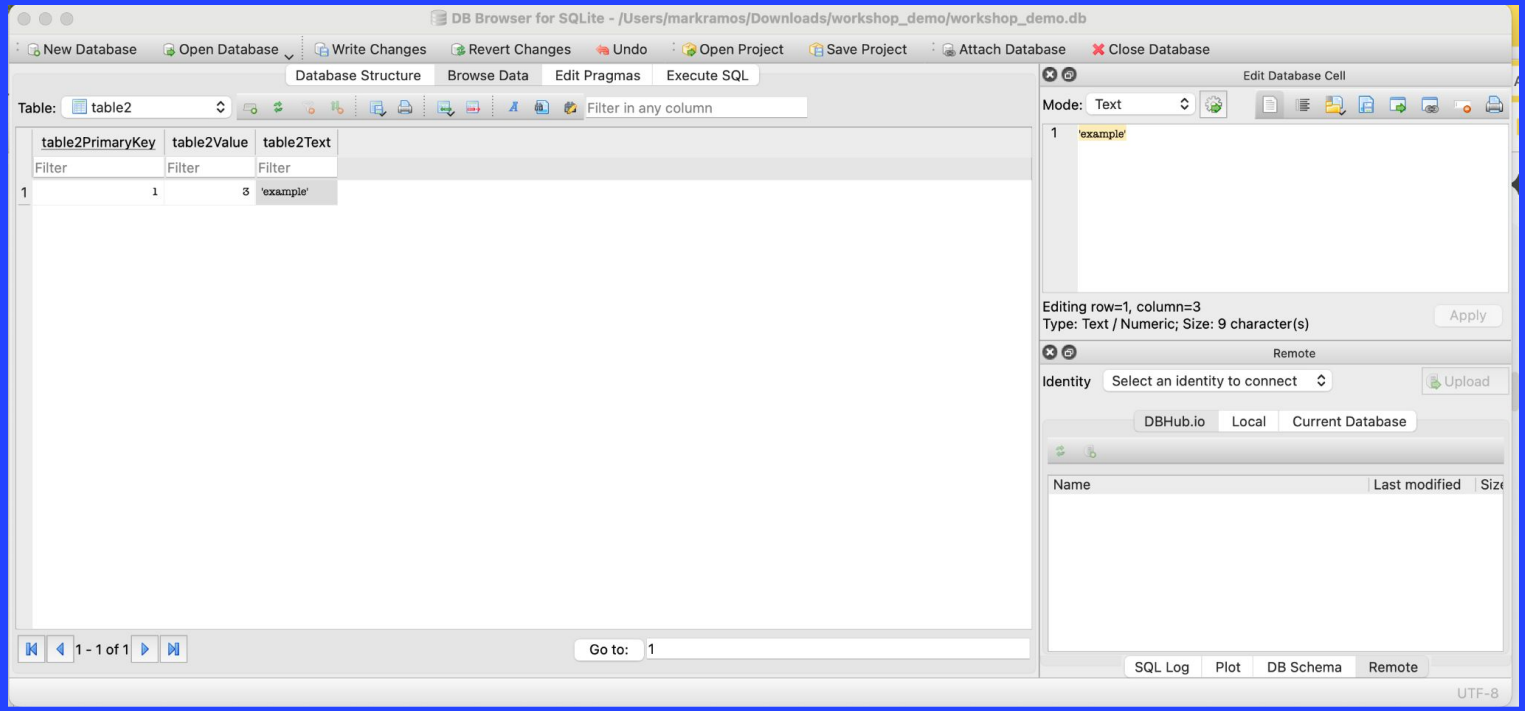
```
sqlite> INSERT INTO composer (composerFirstName,
composerLastName)
   ...> VALUES ('Ludwig', 'Beethoven');

sqlite> INSERT INTO tune (composerId, tuneName,
length, dateRecorded)
   ...> VALUES (1, 'Moonlight Sonata', 15, '1801');
```

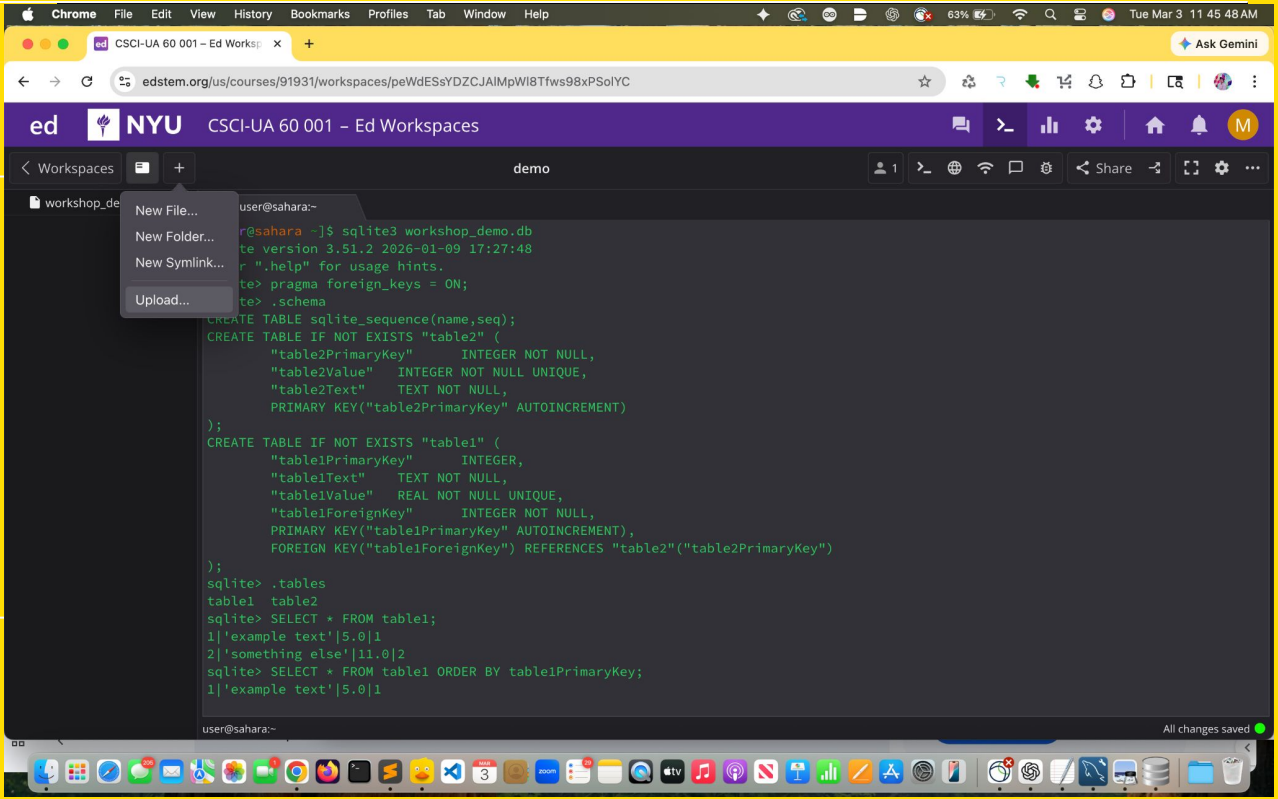
DBSqlite
INSERT



DBSqlite:
INSERT



If you used DB4SQlite your populated database should now be ready to import into edstem



Verify your structure:

01

```
sqlite> .schema
CREATE TABLE sqlite_sequence(name,seq);
CREATE TABLE IF NOT EXISTS "table2" (
"table2PrimaryKey"INTEGER NOT NULL,
"table2Value"INTEGER NOT NULL UNIQUE,
"table2Text"TEXT NOT NULL,
PRIMARY KEY("table2PrimaryKey" AUTOINCREMENT)
);
CREATE TABLE IF NOT EXISTS "table1" (
"table1PrimaryKey"INTEGER,
"table1Text"TEXT NOT NULL,
"table1Value"REAL NOT NULL UNIQUE,
"table1ForeignKey"INTEGER NOT NULL,
PRIMARY KEY("table1PrimaryKey" AUTOINCREMENT),
FOREIGN KEY("table1ForeignKey") REFERENCES
"table2"("table2PrimaryKey")
);
```

02

```
sqlite> .tables
table1  table2
```

clause: ORDER BY

ORDER BY controls the order in which rows are returned in a query result.

ORDER BY does not change the data in a table. It only changes how the result is presented.

Can view:

- ASC - Ascending order (default): smallest to largest, A → Z, Oldest → Newest
- DESC - Descending order: largest to smallest, Z → A, Newest → Oldest

View your data and filter view:

- sqlite> SELECT * FROM table1;
- sqlite> SELECT table1PrimaryKey, table1Text FROM table1
ORDER BY table1PrimaryKey DESC;
- sqlite> SELECT * FROM table1 ORDER BY table1PrimaryKey ASC, table1Text DESC;

01

1. Sort composers alphabetically by last name
2. Sort composers Z–A
3. Sort tunes by recording date (newest first)
4. Sort tunes by length (shortest to longest)
5. Show composers sorted by last name, then by first name.
6. Sort tunes by composerId, then by tune name
7. BONUS: Show the 3 longest tunes by using LIMIT 3 at the end of your SELECT/ORDER BY statement

JOIN QUERIES



```
SELECT
    table1.columnName,
    table2.columnName
  FROM table1
  JOIN table2
    ON table1.table1ForeignKey =
table2.table2PrimaryKey;
```

The basic JOIN keyword performs an INNER JOIN. The INNER JOIN selects records that have matching values in both tables.

Example:

composer

composerId	composerFirstName	composerLastName
1	Ludwig	Beethoven
2	Claude	Debussy

tune

tuneId	composerId	tuneName
1	1	Moonlight Sonata
2	2	Clair de Lune
3	1	Für Elise

```
SELECT
  tune.tuneName,
composer.composerFirstName,
 composer.composerLastName
FROM tune
JOIN composer
  ON tune.composerId =
composer.composerId;
```

tuneName	composerFirstName	composerLastName
Moonlight Sonata	Ludwig	Beethoven
Clair de Lune	Claude	Debussy
Für Elise	Ludwig	Beethoven

SQL JOIN logic:

TRY THIS ON YOUR OWN TABLES

```
SELECT
  tune.tuneName,
composer.composerFirstName,
 composer.composerLastName
FROM tune
JOIN composer
  ON tune.composerId = composer.composerId;
```

01

Start with the selected table (tune)
It reads each row of the tune table — one song at a time.

02

For each tune, look up the matching composer
It checks the composerId in tune and looks for the same composerId in the composer table.

03

When it finds a match, it merges the rows temporarily
This merged row includes columns from both tables — but only for rows where the composerId values match.

03

Output only the columns listed in the SELECT
It returns tuneName, composerFirstName, and composerLastName.

JOIN Challenges:

1. Add another column to the rows returned: Modify the query to also show the tune's dateRecorded.
2. Change the order of tables: Start the query from the composer table instead.
3. Add a WHERE filter: Show only tunes recorded after a certain date.
4. Sort or group results: Order the output alphabetically by the composer's last name.
5. Write a joining query to find all composers who have more than one tune (add data that matches this criteria to your tables if you don't have any).

SUBQUERY-JOIN- UPDATE

A subquery is a query nested inside another query.

```
SELECT *
FROM table1
WHERE columnName = (
    SELECT columnName
    FROM table2
);
```

```
UPDATE table1
SET columnName = value
WHERE primaryKey = (
    SELECT foreignKey
    FROM table2
    WHERE condition
);
```

- Update one table based on another.
- Avoid manually finding IDs.
- Maintains data consistency.
- Great for batch updates and cleaning related data.

Example:

```
UPDATE composer
SET composerLastName = 'Beethoven'
WHERE composerId = (
SELECT composerId
FROM tune
WHERE tuneName = 'Moonlight Sonata'
);
```

- UPDATE composer → identifies the table whose rows will be modified.
- SET composerLastName = ‘Beethoven’ → specifies the new value that will be written to the column.
- WHERE composerId = (subquery) → determines which row in composer should be updated.
- SELECT composerId FROM tune WHERE tuneName = ‘Moonlight Sonata’ → the subquery finds the composerId associated with that tune, and the outer query updates the matching composer.

composer

composerId	composerFirstName	composerLastName
1	Ludwig	Beethoven
2	Claude	Debussy

tune

tuneId	composerId	tuneName
1	1	Moonlight Sonata
2	2	Clair de Lune
3	1	Für Elise

Try it yourself!:

01

```
UPDATE composer
SET composerLastName = '<<one of your
artists>>'
WHERE composerId = (
  SELECT composerId
  FROM tune
  WHERE tuneName = '<<one of your songs>>'
);
```